VEGA

Autonomous System Controller

Project Report | v1.0 | 1 September 2026

A local-first autonomous assistant that operates the machine it runs on. It manages work, files and documents, drives a web browser, reports on the system, sends email, and takes instructions by voice. Every model runs locally; no user data reaches a hosted service.

> **Scope of this document.** This is an overview intended to convey what the system does and how it is organised. It is deliberately not a detailed engineering report: implementation specifics, internal algorithms and the development approach are not disclosed here. Where a figure is given it is measured rather than estimated, and the limitations in section 5 are stated as plainly as the capabilities.

Structure	11 capability agents, 23 core modules, 6 learning modules
Model interface	9 tools exposing 70 distinct actions
Command surface	78 recognised intent types
Verification	514 automated tests, 4 standing audits, and a 45-task frozen benchmark
Models	A small reasoning model, a faster fallback tier, a vision model, and an embedding model for memory recall — all local

1. What it does

The system operates across the domains below. Every capability was verified against the real state of the machine rather than against a function's reported result.

Domain	Scope
Work management	Tasks, projects and team records
File management	Files and folders, including opening and organising them
Document generation	Word documents and spreadsheets
Web automation	Navigation, search within a site, and form filling
System management	Hardware, processes, storage and resource reporting
Vision	Screen capture and on-screen text recognition
Communication	Email
Voice	Spoken commands and spoken responses
Memory	Recall of previous instructions and outcomes

2. How it works

A command passes through five stages. The organising principle is that **understanding** an instruction is the hard part and belongs to a language model, while **carrying it out** is mechanical and should not involve one.

1. **Direct recognition.** Familiar instructions are understood immediately, without a model.
2. **Model reasoning.** Anything unfamiliar goes to the LLM Council, which interprets it.

3. **Safety review.** Instructions that could damage the machine are reviewed by an independent component before anything runs.
4. **Execution.** The responsible agent performs the action and confirms it actually happened.
5. **Learning.** The outcome is remembered, so related instructions are handled better later.

Why most instructions never reach a model

A local language model takes seconds to interpret an instruction and can invent details that were never given. For an unambiguous instruction there is nothing to gain from that: the meaning is already clear, and the cost is delay plus a risk of acting on something imagined. Measured across the benchmark, **roughly seven in ten instructions are handled without a model at all**. The remainder are genuinely ambiguous, conversational, or multi-step, and those are where a model earns its cost. The weakness of this arrangement is stated in section 5.

Safety is judged by target, not by wording

An early version treated any instruction containing a word like "delete" as dangerous, which meant it refused ordinary work such as removing a completed task. Risk is now judged by *what is being acted on*: application records are routine and recoverable, while system folders, drives and configuration are not. A safety mechanism that blocks normal work is one people learn to work around, which leaves them less safe than before.

3. System organisation

The subsystems behind the capabilities, and the purpose each serves.

Reasoning and control

Component	Purpose
LLM Council	Several specialised roles — strategist, executor, analyst, vision and others — each backed by a model, with a fallback chain so a heavily loaded machine degrades to a smaller model rather than failing outright.
Chairman	Independent review of instructions judged destructive. Ambiguous confirmation is treated as refusal rather than consent.
Tool registry	The single definition of what a model is permitted to invoke. Nothing is reachable unless it was deliberately exposed here.
Task decomposer	Breaks a multi-part request into ordered steps.
Autonomy engine	Chooses how an instruction should be carried out and drives multi-step work.

Memory and learning

Component	Purpose
Episodic memory	A record of past instructions and outcomes, searched by meaning rather than by exact wording, so a rephrased request still finds what came before.
Learning subsystem	Adapts routing confidence over time from observed outcomes.

Security and durability

Component	Purpose
Sandboxed execution	Generated code runs under restriction rather than freely, in more than one independent layer, so a weakness in any single layer is not sufficient on its own.
Credential storage	Account credentials are encrypted using the operating system's own facility. Any plaintext found is migrated and removed.

Component	Purpose
Durable writes	Stored data is written so that an interruption — a crash, a power loss, a cancelled operation — cannot leave a half-written or empty file.
Command guarding	Dangerous system commands are refused before execution rather than cleaned up afterwards.

Interface and operations

Component	Purpose
Voice pipeline	Speech recognition and speech synthesis run as a separate local service. The microphone threshold is calibrated each session against the actual room, rather than assuming a fixed level.
Browser control	A real browser under program control, optionally keeping a signed-in profile between sessions.
Background workers	Scheduled reminders and a remote instruction queue run alongside the interactive session.
Operational tooling	Built-in commands check the environment, report live success rates, and identify duplicate or stale stored data before repairing it behind a backup.

4. The governing principle: nothing is reported unless it happened

The most consequential failure in a system that acts on your behalf is not a crash. A crash is visible. The dangerous failure is a confident report of success for something that did not occur — a file "created" that is not on disk, a form "filled" on a page that never loaded, an email "sent" that the mail server refused. Those are only discovered by checking by hand, long afterwards, by which point the user has already acted on the assurance.

This system therefore holds a standing rule: **a function that acts on the world must confirm the effect before reporting success.** A document is re-opened after writing. A form field is read back after filling. A process asked to close is checked to see whether it actually closed. A mail server's rejection is treated as a failed send.

Where an effect genuinely cannot be observed — a keystroke sent to whichever window holds focus, a file handed to the operating system to open — the system says so explicitly instead of implying a confirmation it does not have. Being unable to verify is reported honestly; it is never disguised as success.

The rule is enforced automatically rather than left to discipline. An audit runs alongside the test suite and identifies any function that acts and then reports success without confirming it. **That count is currently zero.**

How correctness is checked

Alongside 514 automated tests, four standing audits run continuously. Each exists because of a specific class of failure the project encountered, rather than as general good practice: one proves every recognised instruction has something to carry it out; one guards against a phrase quietly changing meaning; one confirms malformed input cannot crash the system; and one enforces the rule above. A 45-task benchmark, frozen early, allows any change to be judged by measurement rather than by argument.

5. Known limitations

These are measured rather than estimated, and are stated in the same detail as the capabilities. A limitation a reader discovers for themselves devalues every capability that was claimed.

Multi-step instructions

Instructions combining several dependent steps — where the result of one step feeds the next — succeed less than half the time on the benchmark. Single-step instructions are the reliable path. Improving this requires

carrying intermediate state through a plan, which is a structural change rather than a correction.

Understanding does not generalise

Familiar instructions are recognised by rule, which means an unanticipated phrasing is not understood directly and falls through to a model. The system does not currently learn new phrasings on its own. This is the most significant limitation in the project and the first item of planned work.

Generated content is limited by model size

Running entirely on local hardware constrains model size, and that constrains what can be written. Explanatory prose is produced reliably; working code is not. Rather than write unusable output into a user's file, the system detects this and reports an honest failure naming the cause.

Conversational reference

Referring back to something mentioned earlier — "now open that one" — is not yet supported. The design exists but was deliberately deferred beyond this release.

6. Planned work

Priority	Item	Reason
1	Learned instruction understanding	Removes the generalisation ceiling described in section 5
2	Multi-step planning with state	The largest measured capability gap
3	Conversational reference	Small, well-defined, and noticeably improves everyday use

A note on completeness. This release reached its current state through repeated auditing and testing, and every capability described here was confirmed working. It should still be assumed to contain faults that have not yet been encountered. Software of this scope always does, and the most serious defect found so far was discovered not by any automated check but by a person using the system normally and noticing that it had stopped responding. Reports of unexpected behaviour are expected and welcome.

VEGA — Autonomous System Controller — v1.0 — prepared for review.